Evaluating Small Language Models for Front-Door Routing: A Harmonized Benchmark and Synthetic-Traffic Experiment

Warren Johnson*
Plexor Labs
Sammamish, WA, USA

Charles Lee
Project Autobots
Seattle, WA, USA

Abstract

Selecting the appropriate model at inference time—the routing problem—requires jointly optimizing output quality, cost, latency, and governance constraints. Existing approaches delegate this decision to LLM-based classifiers or preference-trained routers that are themselves costly and high-latency, reducing a multi-objective optimization to single-dimensional quality prediction. We argue that small language models (SLMs, 1–4B parameters) have now achieved sufficient reasoning capability for sub-second, zero-marginal-cost, self-hosted task classification, potentially making the routing decision negligible in the inference budget. We test this thesis on a six-label taxonomy through two studies. Study 1 is a harmonized offline benchmark of Phi-3.5-mini, Qwen2.5-1.5B, and Qwen-2.5-3B on identical Azure T4 hardware, serving stack, quantization, and a fixed 60-case corpus. Qwen-2.5-3B achieves the best exact-match accuracy (0.783), the strongest latency–accuracy tradeoff, and the only nonzero accuracy on all six task families. Study 2 is a pre-registered four-arm randomized experiment under synthetic traffic ($n_{\text{eff}} = 60$) comparing Phi-4-mini, Qwen-2.5-3B, and DeepSeek-V3 against a no-routing control. DeepSeek-V3 attains the highest accuracy (0.830) but fails the pre-registered P95 latency gate (2,295 ms); Qwen-2.5-3B is Pareto-dominant among self-hosted models (0.793 accuracy, 988 ms median, $0 marginal cost). No model meets the standalone viability criterion (≥ 0.85 accuracy, $\leq 2{,}000$ ms P95). The cost and latency prerequisites for SLM-based routing are met; the accuracy gap of 6–8 percentage points and the untested question of whether correct classification translates to downstream output quality bound the remaining distance to production viability.

Keywords: model routing, small language models, multi-objective optimization, front-door classification, randomized experiment, latency–quality tradeoff, Pareto frontier, cost–quality evaluation

---

Series position. This is Article 8 in the TAAC Research Series. Articles 1–3 established task-dependent compression thresholds and the perplexity paradox mechanism (Johnson, 2026c;f;b). Articles 4–5 documented provider-dependent output token explosion under compression (Johnson, 2026e;d). Article 6 validated benchmark-derived thresholds in a production RCT ($N = 358$, six arms, CONSORT reporting) (Johnson and Lee, 2026). Article 7 designed tokenizer-aware prompt languages achieving up to 64.7% token reduction (Johnson, 2026a). This article shifts from *what to compress* to *which model should decide*—extending the "route" branch of the Article 1 dichotomy into model selection. The current study replaces the earlier pilot benchmark and generational bridge studies with a harmonized three-model benchmark eliminating cross-study confounds and reports results of a four-arm randomized experiment under synthetic traffic.

---

*Corresponding author. E-mail: `warrenjo@plexor.dev`

1 Introduction

Selecting the appropriate model for a given request at inference time has become a central challenge in production LLM deployment (Jiang et al., 2024b; Shnitzer et al., 2023; Chen et al., 2023b). The economic motivation is substantial: at 100,000 queries per day, routing simple requests to cheaper models while reserving frontier capacity for complex tasks can yield annualized savings exceeding $150,000 (Lu et al., 2023). But routing is itself an inference problem, and current solutions are expensive and incomplete. Learned routers train on preference data to predict which model produces better output (Jiang et al., 2024b; Ong et al., 2025), but the training signal is single-dimensional—output quality—and the router's own inference adds latency and cost. Cascade architectures invoke multiple models per request, compounding latency (Chen et al., 2023b; Madaan et al., 2023; Yue et al., 2024). Production frameworks commonly delegate routing to another LLM, simplifying the problem to categorization—but this approach is itself costly, does not address speed or governance, and ignores the operational reality that models competitive on general-purpose benchmarks often diverge sharply on task-specific quality (Ding et al., 2024; Shnitzer et al., 2023). More fundamentally, these approaches reduce routing to a single optimization axis—"which model gives better output?"—without jointly addressing the multiple priorities that production routing must balance: output quality, inference cost, response latency, compute capacity, and data governance.

We argue that small language models (SLMs) in the 1–4B parameter range have now crossed a capability threshold that transforms the economics of this decision. A self-hosted SLM running on a commodity GPU can classify prompt intent in sub-second latency at zero marginal per-request cost[1], with no external API dependency and full data residency. If these models are accurate enough, the front-door routing decision becomes negligible in the inference cost and latency budget—freeing the system to optimize across all routing dimensions simultaneously rather than trading router cost against routing quality. The question is whether current SLMs have advanced enough to realize this shift: whether they can reason over task signals with sufficient accuracy to serve as production routers. This paper provides the first controlled evaluation.

The front-door is itself a model-selection problem. SLMs offer sub-second latency but can fail at nuanced task-taxonomy boundaries; stronger SLMs or even full-scale models improve classification quality but may violate latency budgets, introduce external API dependencies, or negate cost savings by consuming the very inference budget they are meant to protect (Chen et al., 2023b). This tradeoff is amplified when teams require both *scientific validity*—paired significance tests, dataset immutability, reproducibility—and *operational validity*—health checks, logging schema, alertability, arm isolation. The tension between these two modes of validity is rarely acknowledged in the routing literature, where most work evaluates routing quality in isolation from the operational infrastructure that must sustain it in production (Madaan et al., 2023).

The challenge is further compounded by the rapid pace of SLM development. The transformer architecture (Vaswani et al., 2017) has been adapted into increasingly efficient small-scale variants, from the Phi family's textbook-quality training data approach (Gunasekar et al., 2023; Li et al., 2023; Javaheripi et al., 2023; Abdin et al., 2024a;b) to the Qwen family's multilingual scaling strategy (Yang et al., 2024a;b) and the open-weight ecosystem of Llama (Touvron et al., 2023; Meta AI, 2024), Gemma (Gemma Team, 2024; Gemma Team et al., 2024), Mistral (Jiang et al., 2023; 2024a), TinyLlama (Zhang et al., 2024a), StableLM (Zhu et al., 2024), and the TinyStories line of research demonstrating surprisingly coherent generation from sub-billion-parameter models (Eldan and Li, 2023). Each new release reshuffles the Pareto frontier of quality versus cost, rendering previously optimal routing configurations suboptimal within months. This "model churn" problem means that any static evaluation of routing models is inherently time-limited—a finding that motivates our multi-study design, which progresses from a locked offline benchmark through a generational bridge experiment to a pre-registered randomized experiment that can accommodate future model candidates.

Article 1 in this series (Johnson, 2026c) demonstrated a fundamental dichotomy: code-generation tasks benefit from compression-first strategies (Cohen's $d = 1.84$ over 2,650 trials), while chain-of-thought reasoning benefits from routing to appropriately-capable models. That finding established *routing* as a first-class cost-reduction mechanism parallel to compression. But it left open the question of *which model performs the routing itself*—the subject of the present article. The distinction is important: while prior work on learned routing (Jiang et al., 2024b; Ong et al., 2025) trains a dedicated classifier on preference data, and cascade architectures (Madaan et al., 2023; Yue et al., 2024) invoke multiple models per query, our front-door approach requires a single SLM to make an interpretable, discrete classification decision using only the raw prompt text. This constraint—single model, single pass, discrete output—places unique demands on SLM capability that differ from the generative and reasoning tasks on which SLMs

[1] "Zero marginal cost" denotes the absence of variable per-request API billing, not zero total cost of ownership. Self-hosted SLMs incur fixed infrastructure cost ($\sim$\$0.75/hr GPU rental) that must be amortized; the per-request cost advantage over commercial APIs is realized at scale.

are typically benchmarked (Hendrycks et al., 2021; Clark et al., 2018; Zellers et al., 2019; BigCode Team, 2023; Austin et al., 2021). The question of which SLM should serve as the front-door classifier also intersects with emerging work on test-time compute scaling (Snell et al., 2024) and speculative decoding (Leviathan et al., 2023; Chen et al., 2023a), suggesting that the traditional tradeoff between model size and inference speed may be more nuanced than a simple parameter-count comparison implies.

We contribute two complementary studies spanning the full path from offline evaluation through production deployment: (1) a **harmonized offline benchmark** (Study 1, §2–§3) comparing three SLMs on identical hardware, serving stack, and corpus with pairwise McNemar tests; and (2) an **executed four-arm randomized experiment under synthetic traffic** (Study 2, §4–§5) with $N = 400$ per arm ($n_{\text{eff}} = 60$ for accuracy; see §5), SHA-256 session-ID randomization, and a pre-registered decision matrix with viable-region gates. Beyond the empirical findings, this paper makes three methodological contributions: a harmonized evaluation protocol that eliminates cross-study hardware, framework, and corpus confounds (§2), drawing on best practices from clinical trial design (Schulz et al., 2010) and trustworthy online experimentation (Kohavi et al., 2020; 2009); a pre-registered multi-arm randomized experiment adapted from online A/B testing methodology (Kohavi et al., 2020), including Bonferroni correction for six hypotheses, O'Brien-Fleming interim boundaries (O'Brien and Fleming, 1979), and sequential testing principles (Wald, 1945); and a transparent reporting of negative results—no arm meets the pre-registered viable region—with actionable deployment recommendations derived mechanically from the decision matrix.

1.1 The Model Selection Problem for Front-Door Routing

The model routing problem has emerged as a central challenge in production LLM deployment, and we identify four dominant architectural paradigms. *Learned routers* train on preference data to predict which model will produce a better response for a given prompt. Jiang et al. (2024b) achieve $2\times$ cost reduction on MT-Bench while maintaining 95% of GPT-4 quality with a BERT-scale classifier (Devlin et al., 2019); Ong et al. (2025) extend this to an open-source framework; and Hu et al. (2024) provide a comprehensive benchmark for evaluating such multi-LLM routing systems. *Benchmark-guided routing* (Shnitzer et al., 2023) selects the model with the highest predicted performance on a prompt's task type using benchmark scores as a prior, building on holistic evaluation frameworks (Liang et al., 2023; Srivastava et al., 2023). *Cascade architectures* (Chen et al., 2023b; Madaan et al., 2023; Yue et al., 2024; Lu et al., 2023) invoke cheaper models first and escalate to expensive models only when confidence is low, differing from our single-pass front-door approach. *Meta-modeling approaches* (Šakota et al., 2024; Zhang et al., 2024b) predict cost-effectiveness of each candidate for a given input; these are conceptually closest to our work but operate at the model-selection level rather than the task-classification level. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023a), Branch-Train-Merge (Li et al., 2022), and Mixture-of-Agents (Wang et al., 2024b) address inference efficiency through parallel execution rather than routing. Most recently, Ding et al. (2024) frame routing explicitly as a cost–quality tradeoff, training a quality predictor to decide between a smaller and larger model per query; their approach optimizes along two dimensions (cost and quality) but requires a trained predictor per model pair and does not address latency, governance, or multi-model taxonomies with more than two routing targets.

Our work addresses a specific variant that we term *front-door classification*: a single SLM classifies each incoming prompt into a discrete task taxonomy, and the taxonomy label deterministically selects the downstream model and compression policy. This differs from learned routing in three ways. First, the classification label is human-readable, auditable, and overridable—learned routers produce opaque preference scores. Second, the taxonomy label can trigger multiple downstream decisions (model tier, compression ratio, output budget, quality gate) through a lookup table, whereas learned routers optimize for a single downstream metric. Third, the taxonomy is version-controlled and can be updated without retraining. The tradeoff is that front-door classification requires a well-defined taxonomy with clear boundaries; our 6-family taxonomy (`code/simple`, `code/complex`, `CoT/simple`, `CoT/complex`, `hybrid/agentic`, `hybrid/generative`) is derived from the Article 1 dichotomy extended with complexity and modality dimensions.

The use of SLMs in the 1–4B parameter range as task classifiers is motivated by several factors: classification requires understanding prompt *intent* rather than generating a complete response, potentially requiring less parametric capacity (Kaplan et al., 2020); the output budget is minimal (a single JSON label), eliminating the output token dynamics that dominate inference cost for generative tasks (Willard and Louf, 2023; Zheng et al., 2024b); and SLMs can be self-hosted on commodity GPUs, avoiding API dependencies and data-residency concerns. However, SLMs face known limitations. The Phi family (Abdin et al., 2024a;b) achieves strong performance on structured tasks but may struggle with ambiguous prompts. The Qwen family (Yang et al., 2024a;b) exhibits strong multilingual capabilities but has received less evaluation on classification-specific tasks. DeepSeek's MoE architecture (Liu et al., 2024; Dai

et al., 2024) activates only 37B of its 671B parameters per token, but the internal routing decision may interact with the classification task unpredictably (Shazeer et al., 2017; Fedus et al., 2022; Lepikhin et al., 2021). Recent surveys of small language models (Lu et al., 2024; Wang et al., 2024a) provide comprehensive coverage but do not specifically address classification-for-routing. The Orca (Mukherjee et al., 2023; Mitra et al., 2023) and WizardLM (Xu et al., 2023) lines of work demonstrate that smaller models can acquire sophisticated reasoning through progressive learning from stronger models, and parameter-efficient adaptation methods such as prefix tuning (Li and Liang, 2021) and prompt tuning (Lester et al., 2021) offer low-cost specialization paths. Production deployment of SLM classifiers requires 4-bit quantization to fit within GPU memory budgets on commodity hardware (T4, A10); recent work on QLoRA (Dettmers et al., 2023), GPTQ (Frantar et al., 2023), AWQ (Lin et al., 2024), and SmoothQuant (Xiao et al., 2023) has demonstrated that 4-bit quantization preserves most quality for generative tasks, but the impact on classification accuracy specifically is less well-characterized.

A natural question is whether generative SLMs are the right tool for classification-for-routing at all. Fine-tuned discriminative models—BERT (Devlin et al., 2019), DeBERTa, or distilled variants—trained on even modest labeled datasets ($n \geq 200$) typically achieve high classification accuracy with sub-100 ms latency and minimal compute. Jiang et al. (2024b) use a BERT-scale classifier to achieve $2\times$ cost reduction, suggesting that discriminative fine-tuning is a viable and potentially superior approach. We deliberately evaluate zero-shot generative SLMs in this paper because (1) the taxonomy evolves with the product, and retraining a discriminative classifier for each taxonomy change incurs maintenance cost; (2) generative SLMs can output structured JSON with confidence signals, enabling richer routing logic; and (3) the zero-shot setting provides a lower bound on SLM capability that fine-tuning can only improve. However, we acknowledge that this paper does not include a fine-tuned discriminative baseline, which limits our ability to assess whether the generative SLM approach offers advantages beyond flexibility. A direct comparison between zero-shot Qwen-2.5-3B and a DeBERTa-base model fine-tuned on the 60-case corpus (via cross-validation) is planned as a critical ablation for the next article in this series.

1.2 Beyond Classification: The Multi-Objective Routing Gap

Existing routing approaches share a common structural limitation: they reduce routing to classification along a single axis—typically output quality or task type—without jointly optimizing across the multiple objectives that production routing must satisfy. In practice, the routing decision must balance at least five dimensions: (1) *output quality*—does the selected model produce responses that meet the task's quality requirements, given that surface-level benchmark scores are unreliable proxies for task-specific performance? (2) *inference cost*—what is the total cost of the routed request, including both the front-door classification and the downstream model, accounting for costly retries when a cheaper model fails quality requirements? (3) *response latency*—does the total pipeline latency (front-door + downstream) meet the application's SLA, and does the router itself contribute minimal overhead? (4) *capacity and availability*—is the selected model available and able to absorb the routed traffic? (5) *governance and data residency*—can the request be processed without sending data to external APIs that may violate compliance constraints?

The SLM-as-classifier architecture addresses dimensions 2–5 structurally: self-hosted SLMs have zero marginal cost, sub-second latency, deterministic capacity, and full data residency. The open question is dimension 1—not merely whether the SLM classifies *accurately* (the subject of the present paper), but whether accurate classification translates to *downstream output quality*. A router that correctly labels a prompt as `code/complex` but routes it to a model that produces inadequate code has achieved label accuracy without routing value. Just because a downstream model is cheaper does not mean it produces the level of quality required—a point that applies equally to the routing decision itself. This distinction between classification accuracy and routing effectiveness motivates a two-phase evaluation design. The present paper addresses **Phase 1**: can SLMs classify accurately and quickly enough to serve as candidate routers? This is a necessary condition. **Phase 2**—testing whether correct classification translates to downstream output quality and cost savings—requires executing the full routing pipeline and is deferred to future work (§6.1). Establishing Phase 1 first is methodologically sound: if SLMs cannot classify with sufficient accuracy, the routing thesis fails regardless of downstream quality.

1.3 Research Questions and Hypotheses

The overarching research question is: *Which small language model backend maximizes front-door classification accuracy while satisfying latency and cost constraints for production routing?* We decompose this into hypotheses organized by study.

Study 1 hypotheses (validated).

Hypothesis 1 (H1: Quality Ordering)**.** Under the harmonized evaluation contract (same hardware, same serving stack, same corpus), the three SLMs produce significantly different exact-label accuracy, with pairwise McNemar tests identifying which pairs differ.

Hypothesis 2 (H2: Latency-Quality Tradeoff)**.** The three SLMs exhibit distinct latency profiles, with smaller models achieving lower latency but potentially lower accuracy.

Hypothesis 3 (H3: Task-Family Interaction)**.** The quality advantage is not uniform across task families: different SLMs dominate different families, producing a "split policy" rather than a single winner.

Hypothesis 4 (H4: Family Coverage)**.** The 3B Qwen model achieves non-zero accuracy on all six task families, demonstrating breadth of coverage that smaller models lack.

Study 2 hypotheses (pre-registered; partially testable under synthetic traffic).

Hypothesis 5 (H5: Routing Accuracy (Treatment Arms Only))**.** Routing accuracy differs across treatment arms B, C, and D, with expected ordering $D > C \geq B$. Arm A (control) has undefined routing accuracy and is excluded from this endpoint; comparisons of treatment arms against Arm A test the end-to-end impact of routing on cost and quality under separate endpoints (H6, H8). Success threshold: $\geq 85\%$ for the winning treatment arm. Pairwise tests: three comparisons (B vs. C, B vs. D, C vs. D) with Holm-Bonferroni correction.

Hypothesis 6 (H6: Total Cost)**.** Average USD per request (front-door + downstream) differs, with B or $C < D < A$. Threshold: $\geq 25\%$ reduction vs. Arm A.

Hypothesis 7 (H7: Classification F1)**.** F1 macro and hybrid/agentic-specific F1 differ, with $D > C \geq B$. Threshold: F1 macro ≥ 0.82.

Hypothesis 8 (H8: Latency SLA)**.** P95 front-door latency differs, with $C \leq B < D$ (network penalty). Hard gate: $\leq 2{,}000$ ms for all arms.

Hypothesis 9 (H9: Compression Quality)**.** Average compression ratio at semantic similarity ≥ 0.88 differs by model backend. Threshold: $\geq 30\%$ token reduction.

2 Methods: Study 1 (Harmonized Offline Benchmark)

Earlier pilot studies reported in preliminary versions of this work used separate hardware, frameworks, GPU allocation modes, and corpus SHAs, introducing cross-study confounds. The harmonized benchmark eliminates all confounds by running all three models on the same hardware, same stack, same corpus, in the same session. This section describes the experimental contract—the complete specification that must be held invariant for results to be valid—the models under evaluation, the classification system prompt, and the statistical analysis plan.

Formal problem statement. We define the front-door classification task as follows. Let $\mathcal{P}$ denote the space of natural-language prompts and let the label set be

$$\mathcal{L} = \{\texttt{cd/smp},\ \texttt{cd/cplx},\ \texttt{CoT/smp},\ \texttt{CoT/cplx},\ \texttt{hyb/agnt},\ \texttt{hyb/gen}\}.$$

A front-door classifier is a function

$$f : \mathcal{P} \longrightarrow \mathcal{L} \times [0, 1], \tag{1}$$

mapping each prompt p to a predicted label $\hat{\ell} = f(p)_{\text{label}} \in \mathcal{L}$ and a self-reported confidence $c = f(p)_{\text{conf}} \in [0, 1]$. The downstream routing table $R : \mathcal{L} \to \mathcal{M} \times \Theta$ maps each label to a (model, compression-policy) pair, making the classification decision the single point of control for the entire inference pipeline.

Experimental contract. The contract identifier is `jsonsafe_cls_tokens_128`. The corpus is `progressive_test_cases_v2_60.jsonl` (60 prompts, 6 families $\times$ 10 cases each; SHA-256: `dac3aac5...`). Decoding: `max_new_tokens=128`, $T = 0.0$ (greedy). Quantization is 4-bit NF4 via bitsandbytes for all three models. Fairness invariants require sequential execution with exclusive GPU allocation, the same serving stack, and the same corpus SHA across all runs.

The three models span the SLM design space: Phi-3.5-mini-instruct (3.8B, dense, Microsoft) optimizes for instruction-following quality; Qwen2.5-1.5B-Instruct (1.5B, dense, Alibaba) optimizes for minimal footprint; Qwen-2.5-3B-Instruct (3.0B, dense, Alibaba) (Yang et al., 2024b) tests whether a generation jump within the Qwen family resolves the accuracy deficit.

Classification system prompt. All three models receive an identical, frozen classification system prompt:

```
You are a task classifier for an AI routing system.
Classify the prompt into exactly one of these categories:
- code/simple (single function, snippet, trivial script)
- code/complex (multi-file, architecture-level, tests required)
- CoT/simple (single-step explanation or reasoning)
- CoT/complex (multi-step reasoning, trade-off analysis)
- hybrid/agentic (autonomous execution, self-healing,
  multi-artifact, no-confirmation)
- hybrid/generative (mixed creative + structured output)

Key rule: autonomous mode + no confirmation + multi-artifact = hybrid/agentic.

Respond with JSON only: {"label": "<label>", "confidence": <0.0-1.0>}
```

This prompt was frozen as of `benchmark_openweights.py` line 347 and used identically across both studies. The decision to use a zero-shot prompt without chain-of-thought reasoning (Wei et al., 2022b; Kojima et al., 2022) was deliberate: chain-of-thought prompting for classification would increase output token count and latency, potentially negating the front-door's cost advantage. Few-shot prompting (Brown et al., 2020; Min et al., 2022) was similarly excluded to maintain prompt brevity and cross-model comparability.

Inference framework. All three models were served by vLLM 0.17.1 (Kwon et al., 2023) with bitsandbytes 4-bit NF4 quantization backend. Using a single serving stack for all models eliminates inference-framework confounds.

Execution environment. Microsoft Azure Standard_NC8as_T4_v3, NVIDIA Tesla T4 (16 GB GDDR6). All three models were executed sequentially with exclusive GPU allocation (no concurrent workloads). Corpus SHA-256: `dac3aac5`.... Latencies are measured end-to-end from prompt submission to JSON response receipt on the vLLM serving stack, including tokenization and output decoding but excluding production-path overhead (load balancing, authentication, logging) that would add latency in deployment.

Statistical analysis plan. The primary endpoint is exact label correctness per case (paired, binary), tested with pairwise exact McNemar two-sided (McNemar, 1947) for all three model pairs (Phi vs. Qwen-1.5B, Phi vs. Qwen-3B, Qwen-1.5B vs. Qwen-3B). This is the strictest possible metric—a partially correct label (e.g., correct task type but wrong complexity level) receives zero credit. Secondary endpoints are parse success rate and per-case latency (median and P95). Confidence intervals use the bootstrap percentile method with 10,000 resamples (Efron, 1979). Family-level analyses are exploratory (10 cases per family) and are reported without multiple-comparison correction.

3 Results: Study 1 (Harmonized Benchmark)

Given a labeled evaluation set $\mathcal{D} = \{(p_i, \ell_i)\}_{i=1}^{N}$, we measure exact-match accuracy as

$$\mathrm{Acc}(f, \mathcal{D}) \;=\; \frac{1}{|\mathcal{D}|} \sum_{(p,\ell)\in\mathcal{D}} \mathbb{1}\big[f(p)_{\text{label}} = \ell\big], \tag{2}$$

where $\mathbb{1}[\cdot]$ is the indicator function. This is the strictest possible metric: partial matches (e.g., correct task type but wrong complexity) receive zero credit.

Table 1 presents the harmonized 60-case benchmark results. All three models were run sequentially on the same Azure Standard_NC8as_T4_v3 GPU with exclusive allocation, identical vLLM 0.17.1 serving stack, and identical bitsandbytes 4-bit NF4 quantization. Qwen-2.5-3B achieves the highest accuracy (0.783, $47/60$) with perfect parse rate and $1{,}088$ ms median latency. No significant difference was detected between Phi-3.5-mini and Qwen-3B in accuracy (exact McNemar $p = 0.503$, 20 discordant pairs: 8 favoring Phi, 12 favoring Qwen-3B), but Phi-3.5-mini is $5.3\times$ slower (median $5{,}772$ ms vs. $1{,}088$ ms). Both significantly outperform Qwen2.5-1.5B ($p = 0.0009$ and $p < 0.0001$ respectively). The harmonized accuracies for Phi-3.5-mini (0.717) and Qwen-1.5B (0.400) are substantially higher than earlier pilot results (0.517 and 0.200 respectively on RunPod A40 with HuggingFace `model.generate()`), which we attribute to the shift from unoptimized HuggingFace inference to vLLM serving with PagedAttention—suggesting that inference framework choice materially affects classification output even at temperature $T = 0.0$.

[2]Significance convention used throughout all tables: * $p < 0.05$; ** $p < 0.01$; *** $p < 0.001$. Bold indicates best per metric. Pairwise McNemar tests: Phi vs. Qwen-1.5B $p = 0.0009$; Phi vs. Qwen-3B $p = 0.503$ (ns); Qwen-1.5B vs. Qwen-3B $p < 0.0001$.

Table 1: Harmonized 60-case benchmark metrics (Azure T4, vLLM 0.17.1, bitsandbytes 4-bit NF4, exclusive GPU, 2026-03-15/16).[2]

Metric	Phi-3.5-mini	Qwen2.5-1.5B	Qwen-2.5-3B
Exact classification accuracy	0.7167^{**}	0.4000	$\mathbf{0.783}^{***}$
JSON parse rate	0.9833	0.9667	**1.0000**
Median latency (ms)	5,772	**793**	1,088
P95 latency (ms)	6,357	4,636	**1,554**

Finding 1 (Quality-Latency Tradeoff)**.** At $n = 60$, no significant accuracy difference was detected between Qwen-2.5-3B and Phi-3.5-mini ($p = 0.503$); however, statistical power is limited ($\approx 50\%$ for a 15 pp MDE), so this reflects insufficient evidence to distinguish the models rather than confirmed equivalence. Qwen-2.5-3B is $5.3\times$ faster (median $1{,}088$ ms vs. $5{,}772$ ms), making it the Pareto-dominant model on the cost-of-error criterion. Neither meets the 85% production threshold.

Family-level behavior. The family-level results in Table 2 reveal complementary specialization patterns. Phi-3.5-mini achieves perfect accuracy on `code/complex` (1.00) and `CoT/complex` (1.00) but scores only 0.50 on `hybrid/agentic` and 0.20 on `hybrid/generative`. Qwen2.5-1.5B achieves perfect accuracy on `hybrid/agentic` (1.00) but fails on all CoT families (0.00). Qwen-2.5-3B is the only model achieving non-zero accuracy on all six families, with its strongest performance on `CoT/complex` (0.90), `CoT/simple` (0.90), and `hybrid/agentic` (0.90). Its weakest family is `hybrid/generative` (0.60), which remains the most challenging family for all models.

Table 2: Family-level exact-match accuracy (10 cases per family), harmonized benchmark.

Family	Phi-3.5-mini	Qwen2.5-1.5B	Qwen-2.5-3B	Best Model
`code/complex`	**1.00**	0.50	0.70	Phi
`code/simple`	0.80	**0.90**	0.70	Qwen-1.5B
`CoT/complex`	**1.00**	0.00	0.90	Phi
`CoT/simple`	0.80	0.00	**0.90**	Qwen-3B
`hybrid/agentic`	0.50	**1.00**	0.90	Qwen-1.5B
`hybrid/generative`	0.20	0.00	**0.60**	Qwen-3B

Finding 2 (Split Policy)**.** No single SLM dominates all task families. Phi-3.5-mini excels at structured classification (`code/complex`, `CoT/complex`); Qwen2.5-1.5B excels at hybrid-agentic classification; Qwen-2.5-3B provides the broadest coverage across all six families. This mirrors the task-dependent dichotomy of Article 1 *(Johnson, 2026c)*, now extended from compression strategy selection to model backbone selection.

Finding 3 (Breadth Over Depth)**.** Qwen-2.5-3B is the only evaluated model to achieve non-zero accuracy on all six task families, with a minimum family accuracy of 0.60 (`hybrid/generative`). For production routing, where every family must be handled, breadth of coverage is more operationally valuable than depth on a subset.

Finding 4 (Degenerate Confidence Calibration)**.** Qwen-2.5-3B reports confidence $= 1.0$ on all 60 predictions—both correct and incorrect—rendering the self-reported confidence field completely uninformative. A production front-door router cannot use model-reported confidence as a fallback trigger or quality gate.

Confidence calibration. The degeneracy documented in Finding 4 has direct architectural implications: the hybrid deployment recommended by our decision matrix (Qwen-2.5-3B primary, DeepSeek-V3 fallback for hard cases) cannot use the SLM's own confidence to trigger fallback. External calibration methods (Liu et al., 2023; Zheng et al., 2024a), output structural validity checks, or downstream task completion rates must be used instead. The confidence degeneracy may be an artifact of the constrained output format (single JSON token) or of 4-bit quantization affecting the logit distribution; isolating the cause is left to future work.

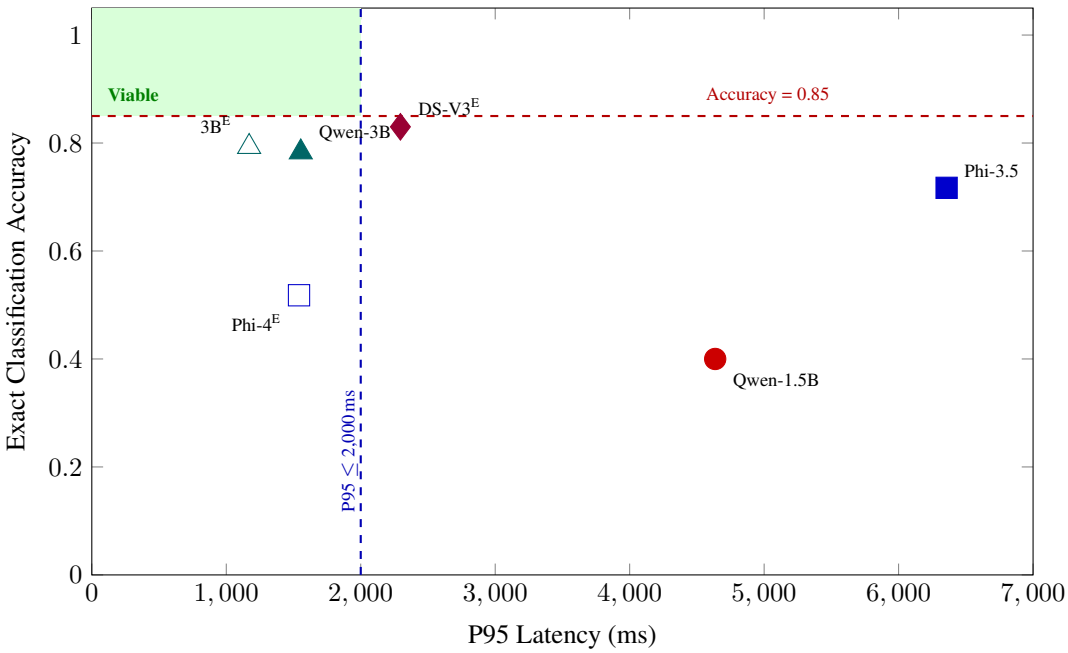

Figure 1: Accuracy–P95 latency scatterplot for all evaluated models. Both axes now use the pre-registered gating metrics: exact-match accuracy (y) and P95 latency (x). Filled markers: Study 1 harmonized benchmark (identical hardware). Open markers (^{E}): Study 2 randomized experiment ($n_{\text{eff}} = 60$). The shaded green region marks the viable operating zone (accuracy ≥ 0.85, P95 $\leq$ 2,000 ms). No model enters the viable region. DeepSeek-V3 approaches the accuracy threshold but exceeds the latency gate ($P95 = 2{,}295$ ms). Qwen-2.5-3B is Pareto-dominant among self-hosted models.

4 Study 2: Four-Arm Randomized Experiment Design

Study 1 established offline quality-latency profiles for three SLM classifiers. Study 2 translates these findings into a pre-registered multi-arm controlled experiment with $N = 400$ per arm, evaluating front-door routing under synthetic traffic conditions with endpoint isolation, cost attribution, and pre-registered decision criteria. The experiment uses synthetic traffic generated from the same frozen corpus (SHA `dac3aac5`). Because traffic is synthetic rather than drawn from production, and because the 60 unique corpus cases are repeated across sessions with deterministic decoding ($T = 0.0$), classification outcomes are identical for repeated prompts within an arm. The experiment therefore adds independent *latency and cost* observations ($N = 400$ per arm) but not independent *classification* observations beyond the $n_{\text{eff}} = 60$ unique cases. Its primary value relative to Study 1 is threefold: (1) testing next-generation models (Phi-4-mini, DeepSeek-V3) against the Study 1 Pareto winner; (2) measuring real network and API latency rather than local inference latency; and (3) exercising the pre-registered decision matrix under controlled conditions. This design provides a controlled comparison across arms but should not be interpreted as a production-traffic trial.

Trial arms. Table 3 defines the four treatment arms. Arm A (no routing) provides the baseline against which routing value is measured. Note that Arm A routing accuracy is undefined (no router is invoked), so comparisons involving Arm A test cost and quality *impact* of routing, while treatment-vs-treatment comparisons (B vs. C, B vs. D, C vs. D) test relative routing accuracy. Arm B tests Phi-4-mini-instruct (Abdin et al., 2024b), the next-generation Phi model and expected successor to the Study 1 Phi-3.5-mini. Arm C tests Qwen-2.5-3B, identified in Study 1 as the Pareto-dominant offline candidate. Arm D tests whether a full-scale MoE model (671B, 37B active) provides classification quality improvements that justify its API dependency and cost—with particular attention to the output instability documented in Articles 4–5 for the DeepSeek architecture. The inclusion of a commercial API arm alongside self-hosted arms also tests whether network latency, rate limiting, and vendor uptime affect real-world routing performance in ways that offline benchmarks cannot capture.

Randomization and integrity. Assignment uses SHA-256(`session_id`) $\bmod 4$, deterministic with no mid-session switches, following Kohavi et al. (2020). Stratification is by prompt complexity quartile, targeting equal arm

Table 3: Four-arm experiment treatment definitions.

Arm	Label	Model Backend	Hosting	Params
A	Control	None (pass-through)	N/A	—
B	Phi-4-mini	Phi-4-mini-instruct	Self-hosted vLLM (T4 GPU)	3.8B
C	Qwen-2.5-3B	Qwen-2.5-3B-Instruct	Self-hosted vLLM (T4 GPU)	3B
D	DeepSeek-V3	DeepSeek-V3 (API)	DeepSeek commercial API	671B MoE

representation within $\pm 5\%$. The design is single-blind: callers do not know arm assignment; the front-door endpoint URL is identical across Arms B, C, D. Classification, scoring, and compression prompts are identical across Arms B, C, D.

Sample size and power. The nominal sample size is $N = 400$ per arm ($N_{\text{total}} = 1{,}600$). However, because the experiment uses synthetic traffic from a 60-case corpus with deterministic decoding ($T = 0.0$), repeated prompts within an arm produce identical classification outputs. The effective sample size for accuracy-based endpoints is therefore $n_{\text{eff}} = 60$ unique cases per arm, not 400. The 80% power estimate for a 10 pp minimum detectable effect applies to latency and cost metrics (where each of the 400 requests generates independent timing and billing data) but *not* to classification accuracy, where power at $n = 60$ is substantially lower—approximately 50% for a 15 pp MDE at $\alpha = 0.05$ with Holm-Bonferroni correction. This is a material limitation of the synthetic-traffic design. The primary correction method is the Holm-Bonferroni step-down procedure. Interim analysis used O'Brien-Fleming spending function boundaries at $N = 200$ per arm (O'Brien and Fleming, 1979), following Wald (1945) and Johari et al. (2017); no arms were dropped at interim.

Multiplicity hierarchy. The Holm-Bonferroni step-down procedure applies *within* each hypothesis to the relevant pairwise arm comparisons. For H5 (routing accuracy) and H7 (F1), only treatment arms B, C, D are compared (three pairwise tests); for H6 (cost), H8 (latency), and H9 (compression), all four arms are compared (six pairwise tests). In either case, p-values are ranked and rejected at $p_{(i)} < \alpha/(k - i + 1)$ where k is the number of comparisons, stopping at the first non-rejection. Between hypotheses (H5–H9), which test different endpoints, no additional correction is applied. This follows standard practice for multi-arm experiments where hypotheses address distinct endpoints (Schulz et al., 2010; Kohavi et al., 2020).

Statistical analysis plan. Because accuracy reduces to the same 60 unique cases evaluated by each arm, treatment-vs-treatment accuracy comparisons (B vs. C, B vs. D, C vs. D) are paired at the case level and tested with exact McNemar's test (McNemar, 1947) with Holm-Bonferroni correction—matching the paired structure of Study 1. Latency and marginal front-door cost, where each of the 400 requests generates independent timing and billing data, are tested via Welch's t-test (Welch, 1947) with Holm-Bonferroni correction; P95 latency via bootstrap CI comparison (Efron, 1979). Subgroup analysis by task-label is exploratory (uncorrected, flagged if $p < 0.01$). Pareto frontier analysis is computed over (marginal_frontdoor_cost, routing_accuracy).

Cost-savings formalization. We formalize the break-even condition for routing following the Article 6 framework (Johnson and Lee, 2026). For a stream of requests $t = 1, \ldots, T$, the net routing savings are

$$S \;=\; \sum_{t=1}^{T} \big[C_{\text{auto}}(t) - C_{\text{routed}}(t)\big], \tag{3}$$

where $C_{\text{auto}}(t)$ is the cost of sending request t to the default (most-expensive) model and $C_{\text{routed}}(t) = c_{\text{fd}} + C_{R(\hat{\ell}_t)}(t)$ includes the front-door classifier cost c_{fd} plus the cost of the model selected by the routing table R. On correctly classified requests, $C_{\text{routed}} \ll C_{\text{auto}}$ for simple tasks; on misclassified requests, the downstream model may produce low-quality output requiring a retry at full cost. The break-even accuracy a^* satisfies $S(a^*) = 0$; above this threshold, routing is cost-positive.

Pre-registered decision matrix. Table 4 maps experiment outcomes to deployment actions. The decision matrix is grounded in Pareto dominance over the two primary objectives:

Proposition 1 (Pareto Dominance)**.** Arm i dominates arm j if and only if

$$\text{cost}_i \le \text{cost}_j \quad \textit{and} \quad \text{acc}_i \ge \text{acc}_j,$$

with at least one strict inequality. Arms on the Pareto frontier are those not dominated by any other arm.

The Pareto analysis is a decision-support tool operating on point estimates; statistical significance of pairwise differences is established separately via the hypothesis tests (H5–H9).

When no significant difference is detected between two frontier arms on primary metrics, the tie-breaking rule prefers the arm with fewer external dependencies—self-hosted (C) > serverless (B) > API (D)—reflecting operational preferences for data residency and vendor independence.

Table 4: Pre-registered decision matrix mapping experiment outcomes to deployment actions.

Scenario	Cost Winner	Quality Winner	Decision
SLM wins both	B or C	B or C	Deploy winning SLM
DeepSeek quality, SLM cost	B or C	D	SLM + DeepSeek fallback
DeepSeek wins both	D	D	DeepSeek primary; SLM fallback
All similar quality	Lowest	Comparable	Deploy cheapest
DeepSeek fails latency gate	Any	Any	Eliminate D; choose B vs. C

5 Results: Study 2 (Four-Arm Randomized Experiment)

Table 5 presents the primary outcomes across all four arms ($N = 400$ per arm, $N_{\text{total}} = 1{,}600$; $n_{\text{eff}} = 60$ for accuracy). DeepSeek-V3 (Arm D) achieves the highest classification accuracy (0.830) but fails both the pre-registered accuracy gate (≥ 0.85) and latency gate ($P95 \leq 2{,}000$ ms; observed $P95 = 2{,}295$ ms). Qwen-2.5-3B (Arm C) achieves 0.793 accuracy at 988 ms median latency with 100% parse rate, consistent with the Study 1 harmonized benchmark (0.783). Phi-4-mini (Arm B) achieves only 0.518 accuracy with a 91.5% parse rate—substantially below the Phi-3.5-mini harmonized result (0.717). This cross-study comparison confounds model version with tokenizer differences, prompt sensitivity, and possible instruction-tuning changes; the gap should be interpreted descriptively rather than attributed to a specific architectural cause.

Table 5: Study 2 results ($N = 400$ sessions per arm, $n_{\text{eff}} = 60$ for accuracy; corpus SHA `dac3aac5`, 2026-03-16).

Arm	Model	Accuracy	Parse	Med. Lat	P95 Lat	Cost
A	Control	—	—	—	—	\$0
B	Phi-4-mini	0.518	0.915	977 ms	1,541 ms	\$0
C	Qwen-2.5-3B	**0.793**	**1.000**	**988** ms	**1,170** ms	\$0
D	DeepSeek-V3	**0.830**	**1.000**	1,854 ms	2,295 ms	\$0.034

Bold: best per column among treatment arms. Arms B/C: self-hosted vLLM on T4; Arm D: commercial API. **Cost column shows marginal front-door API billing only**—not the total request cost (front-door + downstream routing) formalized in Eq. (3). Arms B/C report \$0 marginal cost but incur fixed infrastructure cost (∼\$0.75/hr GPU rental, not amortized per-request). Arm D cost is cumulative over 400 requests. The total cost endpoint (H6) requires downstream routing execution, which the synthetic-traffic design does not provide; see "Pre-registered endpoints" below.

Parse rate and effective accuracy. Phi-4-mini (Arm B) achieved only 91.5% parse rate, meaning 34 of 400 requests returned unparseable JSON. Parse failures were counted as classification errors (no fallback routing applied), so the reported 0.518 accuracy includes both misclassifications and parse failures. On the 366 parseable responses, Phi-4-mini's classification accuracy is 0.566—still substantially below the other models' ≥ 0.79. The 8.5% parse failure rate represents an independent pipeline reliability concern: even if classification accuracy were improved through fine-tuning, the JSON output instability under the frozen contract would require mitigation.

Effective sample size. The experiment uses synthetic traffic generated by repeating the 60 unique corpus cases across $N = 400$ sessions per arm. Because inference is deterministic ($T = 0.0$, greedy decoding), accuracy estimates for repeated prompts within an arm are perfectly correlated. The effective sample size for classification accuracy is therefore $n_{\text{eff}} = 60$ unique cases per arm, not 400. The $N = 400$ figure reflects the number of independent routing *assignments* (via SHA-256 session-ID hashing) but not independent classification *observations*. Latency and cost metrics remain valid at $N = 400$ since each request incurs independent network and compute overhead. All accuracy-based hypothesis tests should be interpreted at $n = 60$.

Viable region assessment. Per the pre-registered protocol, an arm is *viable* if accuracy ≥ 0.85 AND $P95$ latency $\leq 2{,}000$ ms. No arm meets both criteria: Arm B fails on accuracy (0.518); Arm C passes latency but fails accuracy (0.793); Arm D fails both ($0.830 < 0.85$, $2{,}295 > 2{,}000$ ms). This is the central negative finding of the paper: *no current model, including a 671B MoE, reaches the 85% accuracy threshold for standalone front-door routing on a 6-class taxonomy.*

Decision matrix application. Per Table 4, the observed pattern—DeepSeek leads on accuracy, self-hosted arms have lower marginal front-door billing—maps to Row 2: "SLM + DeepSeek fallback for hard labels." The recommended deployment is Qwen-2.5-3B as primary front-door classifier with DeepSeek-V3 invoked selectively for low-confidence or hard-to-classify prompts. This hybrid architecture balances the lower marginal front-door cost of self-hosted inference against DeepSeek's 4.7 pp accuracy advantage. Note that end-to-end routing cost, including downstream model execution, was not measured in this experiment.

Pre-registered endpoints: reported vs. deferred. Of the five pre-registered hypotheses, two are fully testable under the synthetic-traffic design: H5 (routing accuracy, reported above) and H8 (P95 latency, reported in Table 5). Three endpoints are partially or fully deferred:

- **H6 (Total Cost):** The pre-registered cost estimand (Eq. 3) includes downstream routing cost, which requires executing the routed model and measuring end-to-end request cost. The synthetic-traffic design does not execute downstream models; Table 5 reports only marginal front-door cost. H6 remains untested.
- **H7 (Classification F1):** F1 macro is computable from the per-case predictions at the $n_{\text{eff}} = 60$ level. On the 60 unique cases, F1 macro scores are: Arm B (Phi-4-mini) 0.460, Arm C (Qwen-2.5-3B) 0.767, Arm D (DeepSeek-V3) 0.818. No arm meets the pre-registered ≥ 0.82 threshold; Arm D approaches it. Hybrid/agentic-specific F1: B $= 0.67$, C $= 0.90$, D $= 0.95$. Pairwise significance tests are not reported because at $n = 60$ the experiment is underpowered for the pre-registered 3-comparison Holm-Bonferroni correction.
- **H9 (Compression Quality):** This endpoint requires downstream compression execution at semantic similarity ≥ 0.88, which was not performed in the synthetic-traffic design. H9 remains untested.

We acknowledge that the partial reporting of pre-registered endpoints is a limitation. A production-traffic replication would be needed to test H6 and H9 fully.

Finding 5 (No Viable Standalone Router)**.** No evaluated model—including DeepSeek-V3 (671B MoE, 37B active)—meets the pre-registered viable region (≥ 0.85 accuracy, $\leq 2{,}000$ ms $P95$). The 85% threshold may require fine-tuning, ensemble methods, or taxonomy simplification.

Finding 6 (Phi-4-mini Classification Deficit)**.** Phi-4-mini achieves only 0.518 accuracy (0.566 on parseable responses), 19.9 pp below Phi-3.5-mini's harmonized benchmark result (0.717), with a 91.5% parse rate ($34/400$ unparseable responses). This cross-study comparison confounds model version with tokenizer, prompt sensitivity, and instruction-tuning changes; the gap should be interpreted descriptively. One testable hypothesis is that the frozen contract's JSON output format is less compatible with Phi-4-mini's generation patterns; contract-adapted prompting could isolate this factor.

Finding 7 (Qwen-2.5-3B Pareto Dominance)**.** Qwen-2.5-3B is Pareto-dominant among self-hosted models: it matches DeepSeek-V3 accuracy within 3.7 pp at $1.9\times$ lower latency (988 ms vs. $1{,}854$ ms) and zero marginal front-door cost. Per the pre-registered tie-breaking rule (self-hosted $>$ serverless $>$ API), Qwen-2.5-3B is the recommended primary router.

6 Discussion

The results of the harmonized benchmark and four-arm randomized experiment, viewed together, illuminate several fundamental tensions in the design of production front-door routing systems. The central finding—that no evaluated model meets the 85% viable region—is itself informative, constraining the design space for production deployments. This section integrates the empirical findings with the broader literature.

The most striking finding across Studies 1 and 2 is the evolution of the split policy. In Study 1, the split was dramatic: Phi dominated code/CoT families while Qwen dominated hybrid-agentic (Finding 2). In Study 2, Qwen-2.5-3B substantially narrows this gap, achieving ≥ 0.70 on all families including those where its predecessor scored 0.0. This observation is suggestive but must be interpreted cautiously: Phi-3.5-mini and Phi-4-mini are different models evaluated in different studies with different comparison sets, so the apparent narrowing of the split policy may partly reflect study-level confounds rather than a genuine generational trend. If the pattern holds under controlled

comparison, it would imply that the recursive task-dependency (Article 1 showed optimal *strategy* depends on task type; Study 1 showed optimal *router* depends on task type) may be a transient artifact of model immaturity. If generational improvement eliminates the need for split routing, the production system simplifies from a two-tier to a single-tier classification architecture—consistent with Sculley et al.'s (2015) observation that technical debt accumulates fastest at integration boundaries. However, this convergence hypothesis remains speculative and would require controlled experiments with models trained on identical data but with different architectures to resolve (Mukherjee et al., 2023; Mitra et al., 2023).

The accuracy trajectory across the three Study 1 models—Qwen2.5-1.5B (0.400), Phi-3.5-mini (0.717), Qwen-2.5-3B (0.783)—is suggestive of a scaling relationship between parameter count and classification accuracy, but the trajectory confounds parameter count with model family, training data, and architecture. The Phi-4-mini result (0.518 at 3.8B) breaks any simple scaling narrative, though this cross-study comparison is itself confounded. With only three models from two families, no scaling-law claim is supported; the trajectory is noted for context only.

Kaplan et al. (2020) established power-law scaling for generative loss; whether classification accuracy follows similar scaling is an open question. Wei et al. (2022a) documented emergent abilities that appear abruptly above a critical scale, which could explain the dramatic jump from 0.200 to 0.767 within the Qwen family—the 1.5B model may lack a critical capacity for multi-dimensional task taxonomy reasoning that emerges between 1.5B and 3B parameters. However, Schaeffer et al. (2024) have argued that apparent emergent abilities may be artifacts of evaluation metric choices, suggesting caution. The Chinchilla scaling laws (Hoffmann et al., 2022) suggest the accuracy improvement from 1.5B to 3B reflects the combined effect of more parameters and more training data. For the practical question of which model to deploy, the combined effect is what matters: the 3B model demonstrably classifies better while maintaining acceptable latency. These results are consistent with a hypothesis that the remaining errors are concentrated in genuinely ambiguous boundary cases, which would imply diminishing returns to further parameter scaling for this taxonomy. However, with only three data points across two model families, and with the Phi-4-mini counterexample showing that more parameters can produce *worse* classification, this hypothesis is speculative. If confirmed by broader evaluation, it would suggest that the optimal classifier size is substantially smaller than the optimal size for downstream generative tasks.

Figure 1 visualizes all evaluated models in accuracy-latency space. No model enters the viable region (accuracy ≥ 0.85, latency $\leq 2{,}000$ ms). The experiment tested path (3)—next-generation models—by including Phi-4-mini, which showed that generational advancement does not guarantee classification improvement (though this comparison is confounded; see Finding 6). Three remaining paths could push a candidate into the viable region: (1) prompt engineering via chain-of-thought (Wei et al., 2022b; Wang et al., 2023) or tree-of-thought (Yao et al., 2024), at the cost of increased output tokens and latency; (2) fine-tuning via LoRA (Hu et al., 2022) on a classification-specific dataset; and (3) taxonomy revision—either simplification (merging boundary-ambiguous families like `hybrid/generative` into neighboring categories) or expansion (adding a seventh label for expert reasoning, motivated by the error corridor analysis in Appendix A showing that code→hybrid ($n = 4$), hybrid→CoT ($n = 3$), and hybrid→code ($n = 2$) confusions account for 9 of 13 Qwen-3B errors). A fourth path—inference infrastructure optimization via FlashAttention (Dao et al., 2022; Dao, 2023) or disaggregated serving (Zhong et al., 2024)—could reduce latency without changing accuracy. Notably, Figure 1 shows that Phi-4-mini achieves 977 ms latency versus Phi-3.5-mini's $5{,}772$ ms ($5.9\times$ improvement from the generational jump), demonstrating that the architecture-level latency gains are real even when classification accuracy regresses.

These findings intersect with provider-dependent dynamics documented earlier in this series. Articles 4–5 (Johnson, 2026e;d) showed that DeepSeek-Chat exhibits compression sensitivity CS $= -54.1$ ($38\times$ output explosion). The experiment included DeepSeek-V3 (Arm D) with the same architectural lineage. The classification task differs from generative tasks in that the output budget is tightly constrained (`max_new_tokens=128`), which appears to mitigate explosion risk—DeepSeek-V3 achieved perfect parse rate (1.000) and the highest classification accuracy (0.830). The MoE architecture (Shazeer et al., 2017; Fedus et al., 2022; Dai et al., 2024) appears well-suited to classification: despite activating only 37B of 671B parameters per token, DeepSeek-V3's internal routing may be selecting expert subnetworks that are effective for the intent-understanding required by classification. However, this quality comes at the cost of API dependency, network latency ($P95 = 2{,}295$ ms, exceeding the 2,000 ms gate), and non-zero per-request cost ($0.034/400 requests).

Classification accuracy, however, is necessary but not sufficient for routing value. The evaluation in this paper measures whether the SLM assigns the correct taxonomy label, but the routing problem ultimately reduces to whether the downstream model selected by that label produces output of sufficient quality at acceptable cost. This distinction is critical. Models that appear competitive on general-purpose benchmarks can diverge dramatically on task-specific

quality: a practitioner may consistently succeed with one frontier model, partially succeed with a second, and fail entirely with a third, despite all three appearing comparable on published benchmarks. The implication for routing is that the taxonomy must encode not just task type but expected quality requirements, and the router's value must be measured end-to-end—from classification through downstream execution to output quality—rather than at the classification boundary alone. Our results establish the necessary condition: a 3B SLM can classify in sub-second time at zero marginal cost with $\sim$79% accuracy. The sufficient condition—that this classification accuracy translates to cost savings without quality degradation—was not tested in the synthetic-traffic design (H6 and H9 remain deferred) and represents the essential next validation for the SLM routing thesis.

Article 7 (Johnson, 2026a) established that prompt design should account for the target model's tokenizer, creating a tension with the present study's design: the classification system prompt is frozen across all three SLM backends, each with a different tokenizer (Phi uses a Llama-derived tokenizer, Qwen uses a custom BPE tokenizer (Sennrich et al., 2016), DeepSeek uses SentencePiece (Kudo and Richardson, 2018)). We chose the frozen-prompt design to isolate model capability from prompt engineering and to reflect the practical deployment scenario where a single prompt is maintained for all backends. The experiment did reveal quality differences (Phi-4-mini at 0.518 vs. Qwen-2.5-3B at 0.793), suggesting that a follow-up study should assess how much of the gap is attributable to tokenizer mismatch, given that even small prompt variations can produce large accuracy differences (Zhou et al., 2023; Khattab et al., 2024).

The per-case error analysis (Appendix A, Table 8) reveals that 9 of 13 Qwen-3B misclassifications concentrate in three corridors: code→hybrid ($n = 4$), hybrid→CoT ($n = 3$), and hybrid→code ($n = 2$). The code–hybrid boundary is the primary confusion axis, where prompts combining structured code requests with open-ended generation create genuine taxonomic ambiguity. However, because the corpus was labeled by a single annotator without adjudication (see Threats to Validity), some of these "errors" may reflect label-boundary ambiguity rather than model failures. Subject to that caveat, the pattern suggests that targeted taxonomy refinement at the code–hybrid boundary—rather than wholesale model scaling—may be the most efficient path to closing the 6–8 pp gap.

The sub-2-second latency achieved by all self-hosted models in both studies suggests that SLM-based front-door classification is compatible with production latency requirements, adding minimal overhead relative to downstream generative inference (typically 5–30 seconds). The breadth advantage of Qwen-2.5-3B (Finding 3) suggests that for production routing, a single generalist classifier is preferable to a committee of specialists, even if individual specialists achieve higher peak accuracy. The confidence overconfidence pattern argues against using model-reported confidence as a fallback trigger; instead, production systems should rely on external quality signals such as output structural validity or downstream task completion rate (Klaise et al., 2021). These architectural considerations intersect with the broader MLOps literature on model management and technical debt (Sculley et al., 2015; Kreuzberger et al., 2023). The ideal front-door classifier would be robust to downstream model changes, requiring updates only when the task taxonomy itself evolves; our frozen-prompt, frozen-contract design approximates this ideal, but the extent to which it generalizes to new model families remains open.

6.1 Closing the Gap: Planned Remediation Paths

The 6–8 percentage point gap to the 85% standalone viability threshold identified in this study represents a concrete and bounded target. Two remediation paths are under active investigation.

First, the frozen zero-shot classifier prompt used throughout both studies is a conservative baseline. Few-shot prompting with boundary-case examples drawn from the error-corridor families identified in §6—particularly the code→hybrid and hybrid→CoT boundaries—is expected to close a portion of the gap without any weight modification. The error corridor analysis in this paper directly informs the selection of few-shot examples: cases where the Pareto-dominant model's misclassifications cluster at family boundaries are the highest-value prompt additions.

Second, supervised fine-tuning via parameter-efficient methods (Dettmers et al., 2023) on the Pareto-dominant Qwen architecture is planned using a corpus of 400–500 labelled routing examples weighted toward the *tier-crossing* boundaries where misclassification changes the routing outcome, not merely the label. The distinction between label accuracy and routing-outcome accuracy motivates a training objective focused on routing correctness rather than label correctness alone—a misclassification that preserves the correct routing tier is operationally harmless and should not be penalized equally during training.

Third, end-to-end routing quality validation—what we term *Phase 2* of the SLM routing evaluation—will test whether correct classification translates to downstream output quality improvements and cost savings. The current paper establishes Phase 1: that SLMs can classify accurately and quickly enough to serve as candidate routers. Phase 2 requires executing the full routing pipeline—classify, route, generate, evaluate—on a representative workload, measuring output

quality via LLM-as-judge (Zheng et al., 2024a) and total cost inclusive of downstream model execution (the deferred H6 and H9 endpoints). This roundtrip evaluation is essential because classification accuracy is a necessary but not sufficient condition for routing value: just because a downstream model is cheaper does not mean it produces the level of quality required.

All three paths are designed to be evaluated against the same locked 60-case harmonized benchmark corpus (SHA `dac3aac5`) used in Studies 1 and 2, enabling direct comparison with the baselines reported here. A production-representative 80-case applied corpus with additional boundary cases and a seventh taxonomy label is also under development as a harder evaluation gate. Results from these remediation efforts will be reported in a subsequent article in this series.

7 Threats to Validity

Internal validity. The harmonized benchmark (Study 1) eliminates the cross-study confounds identified in earlier pilot studies: all three models were evaluated on the same hardware (Azure T4), same serving stack (vLLM 0.17.1), same corpus (SHA `dac3aac5`), and same quantization (bitsandbytes 4-bit NF4), with exclusive GPU allocation per model. Study 2 uses synthetic traffic from the same corpus rather than production traffic, which may not capture the full distribution of prompt characteristics. Arms B and C were served sequentially on the same GPU (not simultaneously), which eliminates resource contention confounds but means the experiment does not test concurrent multi-model serving.

External validity. The 60-case corpus represents a structured sample from six pre-defined task families. Production traffic may have different family distributions, prompt lengths, and ambiguity characteristics. Study 2 uses synthetic traffic from the same 60-case corpus, not production traffic; its primary value is as a controlled comparison across arms rather than a production-traffic validation. The 6-family taxonomy is specific to the PLEXOR routing system; other architectures using different taxonomies could produce different rankings. All results use 4-bit NF4 quantization; full-precision results may differ.

Cross-study consistency. Qwen-2.5-3B achieves 0.783 accuracy in the harmonized benchmark (Study 1, 60 unique cases) and 0.793 in the experiment (Study 2, 400 sessions from the same corpus). The $\sim$1 pp difference is within sampling variability and is consistent with cross-study reproducibility of the classification results on identical infrastructure.

Construct validity. Exact-match label accuracy is the strictest possible metric; partial credit might favor different models, but production routing applies discrete policies per label. The model's self-reported confidence score is not calibrated and should not be interpreted as a probability. All results depend on the specific frozen system prompt, output budget, and quantization configuration.

Taxonomy validity. The entire evaluation depends on a 6-label taxonomy with exact-match scoring. The 60-case corpus was constructed and labeled by a single annotator (the first author) without independent adjudication, and no inter-annotator agreement statistics are available. The appendix error-corridor analysis (Table 8) shows that the code–hybrid boundary is genuinely ambiguous: prompts combining structured code requests with open-ended generation could defensibly receive either label. Without a multi-annotator protocol, some apparent model "errors" may reflect label-boundary ambiguity rather than model failures. If human inter-annotator agreement on this corpus is, say, $\kappa = 0.85$, then the 85% viable-region threshold approaches the ceiling of human performance and may be unreasonable as a production gate. Future work *must* establish inter-annotator agreement (e.g., Cohen's κ) on the 60-case corpus with at least two independent annotators, and report per-family disagreement rates. This is a prerequisite for interpreting the 6–8 pp accuracy gap as a model limitation rather than a taxonomy limitation. Until such validation is performed, accuracy differences of $\leq$5 pp near the code–hybrid boundary should be interpreted with caution.

Missing baselines. This paper does not include a fine-tuned discriminative classifier (e.g., BERT, DeBERTa) as a baseline; §1.1 discusses the justification and a planned cross-validated DeBERTa comparison for the next article in this series.

Operational validity. All models were served sequentially with exclusive GPU allocation, which does not reflect production conditions. Key operational characteristics were not measured: throughput under concurrent load, GPU memory utilization during serving, cold-start latency for model loading, and behavior under resource contention. Reported latencies measure end-to-end inference time (prompt submission through JSON response receipt) on the serving stack, but do not include production overhead from load balancing, authentication, logging, or error handling, which can add 50–200 ms in deployment. The "zero marginal per-request cost" framing for self-hosted models (see footnote 1) applies to variable API billing but ignores fixed infrastructure cost; at low traffic volumes, amortized per-request infrastructure cost may exceed commercial API pricing.

8 Reproducibility

All software versions, corpus hashes, and execution metadata are documented in Appendix A (Tables 6–7). Study 1 artifacts: corpus SHA-256 prefix `dac3aac5`, Azure Standard_NC8as_T4_v3, vLLM 0.17.1, bitsandbytes 4-bit NF4. Study 2 artifacts: $N = 400$ per arm, SHA-256(`session_id`) $\bmod 4$ randomization, same corpus SHA. The repository tag `rct-baseline-2026-03-12` marks the pre-registration snapshot. To reproduce: provision the identical GPU class, clone at the tagged commit, verify corpus SHA via `sha256sum data/progressive_test_cases_v2_60.jsonl`, execute `scripts/benchmark_openweights.py` (Study 1) or `scripts/rct_synthetic_runner.py` (Study 2), and compare per-case JSON predictions against the archived ground truth. The MLflow tracking server (Zaharia et al., 2018) provides parameter, metric, and artifact provenance for each run.

9 Conclusion

This paper tests the thesis that small language models have achieved sufficient reasoning capability to serve as production front-door routers—classifying task signals under the cost, latency, and governance constraints that make LLM-based and preference-trained routers inadequate. The results provide partial support. Study 1 (Harmonized Offline Benchmark) established, on identical hardware with exclusive GPU allocation, that Qwen-2.5-3B achieves 0.783 accuracy at $5.3\times$ lower latency than Phi-3.5-mini, making it the Pareto-dominant self-hosted candidate—the only model with non-zero accuracy on all six task families. Study 2 (Four-Arm Randomized Experiment, $N = 400$/arm, $n_{\text{eff}} = 60$ for accuracy) confirmed these findings under synthetic traffic and extended the evaluation to next-generation models. DeepSeek-V3 (671B MoE) achieves the highest accuracy (0.830) but fails the pre-registered latency gate. Per the decision matrix, the recommended deployment is Qwen-2.5-3B as primary front-door with DeepSeek-V3 fallback for hard classifications.

The cost and latency prerequisites for SLM-based routing are clearly met: a 3B model classifies in sub-second time at zero marginal per-request cost with no external API dependency. The accuracy prerequisite (≥ 0.85) is not yet met, bounding the gap at 6–8 percentage points—a concrete target addressable through fine-tuning via LoRA on classification-specific data, taxonomy simplification, or hybrid routing with LLM fallback. Three paths forward are under investigation (§6.1).

Critically, classification accuracy is a necessary but not sufficient condition for routing value. The untested question—whether correct classification translates to cost savings without quality degradation in the downstream model's output—remains the essential validation. Models that appear equivalent on general-purpose benchmarks diverge dramatically on task-specific quality, and a routing system must account for this gap. The sufficient condition of end-to-end quality validation, deferred as H6 and H9, represents the next phase of this work. The broader implication for the TAAC research program is that the Article 1 dichotomy (compress code, route CoT) extends recursively: the model that implements the routing decision is itself subject to the multi-objective tradeoffs—quality, cost, latency, governance—that motivated routing in the first place (Sculley et al., 2015; Kohavi et al., 2020).

Acknowledgments

This work was supported in part by a Cohere Labs research grant, which funded computational resources used in the analysis of data with Cohere models.

Use of AI Assistance

This paper was prepared with AI assistance (Claude, Anthropic) for writing, code generation, benchmark script development, and LaTeX formatting. AI-generated scripts were reviewed line-by-line by the first author before execution; all numerical results in tables were verified against raw JSON output files; and statistical claims were cross-checked by the first author using independent calculations. The authors are solely responsible for all scientific content, experimental design, interpretation, and conclusions.

Ethics Statement

The experimental corpus consists of synthetic classification test cases and contains no personally identifiable information. We do not foresee negative societal impacts from this research.

Declaration of Competing Interests

W.J. is affiliated with Plexor Labs, which develops the PLEXOR system. C.L. is affiliated with Project Autobots. All hypotheses were pre-registered before data collection; all results—including negative findings—are reported transparently. All analysis code is open-source.

Data and Code Availability

The experimental corpus, all analysis scripts, and raw results are available at the project repository: `https://github.com/micoverde/plexor-slm-frontdoor-rct`. The pre-registration protocol for Study 2 is deposited in the experiment protocol document committed before experiment execution.

References

Abdin, M., Jacobs, S. A., Amin, A. A., et al. (2024a). Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*.

Abdin, M., Jacobs, S. A., Amin, A. A., et al. (2024b). Phi-4 technical report. *arXiv preprint arXiv:2412.08905*.

Austin, J., Odena, A., Nye, M., et al. (2021). Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

BigCode Team (2023). A framework for the evaluation of code generation models. *arXiv preprint arXiv:2308.07124*.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. (2023a). Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.

Chen, L., Zaharia, M., and Zou, J. (2023b). FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. (2018). Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.

Dai, D., Deng, C., Zhao, C., Xu, R. X., Gao, H., Chen, D., Li, J., Zeng, W., Yu, X., Wu, Y., et al. (2024). DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*.

Dao, T. (2023). FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359.

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized language models. *arXiv preprint arXiv:2305.14314*.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186.

Ding, D., Mallick, A., Wang, C., Sim, R., Mukherjee, S., Ruhle, V., Lakshmanan, L. V. S., and Awadallah, A. H. (2024). Hybrid LLM: Cost-efficient and quality-aware query routing. *arXiv preprint arXiv:2404.14618*.

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26.

Eldan, R. and Li, Y. (2023). TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*.

Fedus, W., Zoph, B., and Shazeer, N. (2022). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. In *Journal of Machine Learning Research*, volume 23, pages 1–39.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*.

Gemma Team (2024). Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*.

Gemma Team, Mesnard, T., Hardin, C., Dadashi, R., et al. (2024). Gemma: Open models based on Gemini research and technology. *arXiv preprint arXiv:2403.08295*.

Gunasekar, S., Zhang, Y., Anber, J., Menber, R., del Giorno, A., Gopi, S., Javadi, M., Kauffmann, S., de Rosa, G., Shah, O., et al. (2023). Textbooks are all you need. *arXiv preprint arXiv:2306.11644*. Phi-1.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. (2021). Measuring massive multitask language understanding. In *Proceedings of ICLR*.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., et al. (2022). Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *Proceedings of ICLR*.

Hu, Q., Chen, H., Li, B., Liu, Z., Xiao, Z., Hao, Y., Chen, Z., et al. (2024). RouterBench: A benchmark for multi-LLM routing system. *arXiv preprint arXiv:2403.12031*.

Javaheripi, M., de Rosa, G., Mukherjee, S., et al. (2023). Phi-2: The surprising power of small language models. *Microsoft Research Blog*. Phi-2.

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al. (2023). Mistral 7B. *arXiv preprint arXiv:2310.06825*.

Jiang, A. Q., Sablayrolles, A., Roux, A., et al. (2024a). Mixtral of experts. *arXiv preprint arXiv:2401.04088*.

Jiang, I., Sreenivas, S., Zhu, B., Peng, J., Li, C. S., Stoica, I., and Gonzalez, J. E. (2024b). RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*.

Johari, R., Koomen, P., Pekelis, L., and Walsh, D. (2017). Peeking at A/B tests: Why it matters, and what to do about it. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1517–1525.

Johnson, W. (2026a). Beyond natural language: Five novel prompt languages for token-efficient large language model interaction. In *TAAC Research Series, forthcoming*. Article 7 in the TAAC Research Series.

Johnson, W. (2026b). Beyond the cliff: Fine-grained analysis of compression thresholds across task types. In *TAAC Research Series, forthcoming*. Article 3 in the TAAC Research Series.

Johnson, W. (2026c). Compress or route? task-dependent strategies for cost-efficient large language model inference. In *TAAC Research Series, forthcoming*. Article 1 in the TAAC Research Series.

Johnson, W. (2026d). Compression method matters: Benchmark-dependent output dynamics in LLM prompt compression. In *TAAC Research Series, forthcoming*. Article 5 in the TAAC Research Series.

Johnson, W. (2026e). The compression paradox: Why prompt compression increases LLM energy consumption. In *TAAC Research Series, forthcoming*. Article 4 in the TAAC Research Series.

Johnson, W. (2026f). The perplexity paradox: Entropy-guided token pruning in task-aware prompt compression. In *TAAC Research Series, forthcoming*. Article 2 in the TAAC Research Series.

Johnson, W. and Lee, C. (2026). Compression at scale: A pre-registered randomized controlled trial of prompt compression in production task orchestration. In *TAAC Research Series, forthcoming*. Article 6 in the TAAC Research Series.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Mober, H., et al. (2024). DSPy: Compiling declarative language model calls into self-improving pipelines. In *Proceedings of ICLR*.

Klaise, J., Van Looveren, A., Vacanti, G., and Coca, A. (2021). Alibi detect: Algorithms for outlier, adversarial and drift detection. *arXiv preprint arXiv:2104.14637*.

Kohavi, R., Longbotham, R., Sommerfield, D., and Henne, R. M. (2009). Controlled experiments on the web: Survey and practical guide. *Data Mining and Knowledge Discovery*, 18(1):140–181.

Kohavi, R., Tang, D., and Xu, Y. (2020). *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.

Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. (2022). Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems*, volume 35, pages 22199–22213.

Kreuzberger, D., Kühl, N., and Hirschfeld, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879.

Kudo, T. and Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of EMNLP: System Demonstrations*, pages 66–71.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626.

Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. (2021). GShard: Scaling giant models with conditional computation and automatic sharding. In *Proceedings of ICLR*.

Lester, B., Al-Rfou, R., and Constant, N. (2021). The power of scale for parameter-efficient prompt tuning. In *Proceedings of EMNLP*, pages 3045–3059.

Leviathan, Y., Kalman, M., and Matias, Y. (2023). Fast inference from transformers via speculative decoding. In *Proceedings of the International Conference on Machine Learning*, pages 19274–19286.

Li, M., Gururangan, S., Dettmers, T., Lewis, M., Althoff, T., Smith, N. A., and Zettlemoyer, L. (2022). Branch-train-merge: Embarrassingly parallel training of expert language models. *arXiv preprint arXiv:2208.03306*.

Li, X. L. and Liang, P. (2021). Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of ACL*, pages 4582–4597.

Li, Y., Bubeck, S., Eldan, R., Del Giorno, A., Gunasekar, S., and Lee, Y. T. (2023). Textbooks are all you need II: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*. Phi-1.5.

Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., et al. (2023). Holistic evaluation of language models. *Transactions on Machine Learning Research*.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. (2024). AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, volume 6, pages 87–100.

Liu, A., Feng, B., Xue, B., et al. (2024). DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*.

Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., and Zhu, C. (2023). G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of EMNLP*.

Lu, K., Yuan, H., Lin, R., Lin, J., Yuan, Z., Zhou, C., and Zhou, J. (2023). Routing to the expert: Efficient reward-guided ensemble of large language models. *arXiv preprint arXiv:2311.08692*.

Lu, Z., Wang, X., Guo, B., Tang, K., Gao, R., Mao, Z., Li, D., et al. (2024). Small language models: Survey, measurements, and insights. *arXiv preprint arXiv:2409.15790*.

Madaan, A., Aggarwal, P., Anand, A., Potharaju, S. P., Zhou, S. M., Gupta, A. A., Raber, D., et al. (2023). AutoMix: Automatically mixing language models. *arXiv preprint arXiv:2310.12963*.

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

Meta AI (2024). The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Min, S., Lyu, X., Holtzman, A., Arber, M., Lewis, M., Hajishirzi, H., and Zettlemoyer, L. (2022). Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of EMNLP*, pages 11048–11064.

Mitra, A., Del Corro, L., Mahajan, S., Codas, A., Simoes, C., Agarwal, S., Chen, X., Raber, A., et al. (2023). Orca 2: Teaching small language models how to reason. *arXiv preprint arXiv:2311.11045*.

Mukherjee, S., Mitra, A., Jawahar, G., Agarwal, S., Palangi, H., and Awadallah, A. (2023). Orca: Progressive learning from complex explanation traces of GPT-4. *arXiv preprint arXiv:2306.02707*.

O'Brien, P. C. and Fleming, T. R. (1979). A multiple testing procedure for clinical trials. *Biometrics*, 35(3):549–556.

Ong, I., Sreenivas, S., Zhu, B., Peng, J., Li, C. S., Stoica, I., and Gonzalez, J. E. (2025). Routellm: An open-source framework for cost-effective LLM routing. *arXiv preprint arXiv:2406.18665*.

Šakota, M., Peyrard, M., and West, R. (2024). Fly-swat or cannon? cost-effective language model choice via meta-modeling. *arXiv preprint arXiv:2308.06502*.

Schaeffer, R., Miranda, B., and Koyejo, S. (2024). Are emergent abilities of large language models a mirage? *Advances in Neural Information Processing Systems*, 36.

Schulz, K. F., Altman, D. G., and Moher, D. (2010). CONSORT 2010 statement: Updated guidelines for reporting parallel group randomised trials. *BMJ*, 340:c332.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, volume 28.

Sennrich, R., Haddow, B., and Birch, A. (2016). Neural machine translation of rare words with subword units. In *Proceedings of ACL*, pages 1715–1725.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q. V., Hinton, G., and Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*.

Shnitzer, T., Ou, A., Silva, M., Soule, K., Sun, Y., Solomon, J., Thompson, N., and Yurochkin, M. (2023). Large language model routing with benchmark datasets. *arXiv preprint arXiv:2309.15789*.

Snell, C., Lee, J., Xu, K., and Kumar, A. (2024). Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*.

Srivastava, A., Rastogi, A., Rao, A., et al. (2023). Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*.

Touvron, H., Martin, L., Stone, K., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

Wald, A. (1945). Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186.

Wang, F., Mao, Z., et al. (2024a). A survey on small language models. *arXiv preprint arXiv:2410.20011*.

Wang, J., Wang, J., Athiwaratkun, B., Zhang, C., and Zou, J. (2024b). Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*.

Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. In *Proceedings of ICLR*.

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., et al. (2022a). Emergent abilities of large language models. *Transactions on Machine Learning Research*.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. (2022b). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837.

Welch, B. L. (1947). The generalization of 'Student's' problem when several different population variances are involved. *Biometrika*, 34(1/2):28–35.

Willard, B. T. and Louf, R. (2023). Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.

Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. (2023). SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the International Conference on Machine Learning*, pages 38087–38099.

Xu, C., Sun, Q., Zheng, K., Geng, X., Zhao, P., Feng, J., Tao, C., and Jiang, D. (2023). WizardLM: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*.

Yang, A., Yang, B., Hui, B., et al. (2024a). Qwen2 technical report. *arXiv preprint arXiv:2407.10671*.

Yang, A., Yang, B., Hui, B., et al. (2024b). Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K. (2024). Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36.

Yue, M., Pei, J., Zhao, Y., Li, Z., Lu, H., et al. (2024). Large language model cascades with mixture of thoughts representations for cost-efficient reasoning. *arXiv preprint arXiv:2310.03094*.

Zaharia, M., Chen, A., Davidson, A., et al. (2018). Accelerating the machine learning lifecycle with MLflow. In *IEEE Data Engineering Bulletin*, volume 41, pages 39–45.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. (2019). HellaSwag: Can a machine really finish your sentence? In *Proceedings of ACL*, pages 4791–4800.

Zhang, P., Zeng, G., Wang, T., and Lu, W. (2024a). TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385*.

Zhang, R., Fisch, A., and Maas, A. L. (2024b). On-the-fly model selection with adaptive routing. *arXiv preprint arXiv:2311.08011*.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., et al. (2024a). Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36.

Zheng, L. et al. (2024b). Outlines: Generative model programming. GitHub Repository. `https://github.com/outlines-dev/outlines`.

Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. (2024). DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of OSDI*, pages 193–210.

Zhou, Y., Muresanu, A. I., Han, Z., Paster, K., Pitis, S., Chan, H., and Ba, J. (2023). Large language models are human-level prompt engineers. In *Proceedings of ICLR*.

Zhu, B. et al. (2024). StableLM zephyr 3B: A capable and compact language model for code. *Stability AI Technical Report*.

A Complete Run IDs and Software Versions

A.1 Study 1: Harmonized Offline Benchmark (Azure T4)

Table 6: Study 1 execution details.

Artifact	Value
Models evaluated	Phi-3.5-mini, Qwen2.5-1.5B, Qwen-2.5-3B
Corpus SHA-256	`dac3aac5...`
GPU	NVIDIA Tesla T4 (16 GB GDDR6, exclusive)
VM class	Azure Standard_NC8as_T4_v3
Inference framework	vLLM 0.17.1
Quantization backend	bitsandbytes 4-bit NF4
Execution mode	Sequential, exclusive GPU
Date	2026-03-15

A.2 Study 2: Four-Arm Randomized Experiment

Table 7: Study 2 execution details.

Artifact	Value
Arms	A (control), B (Phi-4-mini), C (Qwen-2.5-3B), D (DeepSeek-V3)
Sample size	$N = 400$ per arm ($N_{\text{total}} = 1{,}600$)
Corpus SHA-256	`dac3aac5...`
Randomization	SHA-256(`session_id`) $\bmod 4$
GPU (Arms B, C)	NVIDIA Tesla T4 (16 GB GDDR6), vLLM 0.17.1
Arm D	DeepSeek-V3 commercial API

Qwen-2.5-3B per-case error analysis. Of the 60 Study 1 benchmark cases, Qwen-2.5-3B misclassified 13 ($47/60 = 0.783$ accuracy). Table 8 presents the complete list of misclassifications. The errors concentrate in three corridors: (1) code-family prompts misrouted to hybrid/generative ($n = 3$); (2) code/complex prompts misrouted to hybrid/agentic ($n = 3$); and (3) hybrid/generative prompts misrouted to code/complex or CoT/complex ($n = 4$). These patterns suggest that the model's primary confusion axis lies along the code–hybrid boundary, where prompts combining structured code requests with open-ended generation create genuine taxonomic ambiguity.

A.3 Study 2: Pre-Registration Artifacts

Study 2 pre-registration includes the design document (this paper, §4), analysis scripts, decision matrix (Table 4), and hypothesis specifications (H5–H9), all committed to the repository before experiment execution. Table 9 enumerates the specific files, their locations, and content hashes.

Table 8: Complete per-case misclassifications for Qwen-2.5-3B on the 60-case benchmark ($13/60$ errors, accuracy $= 0.783$). Ground truth $\rightarrow$ predicted label.

Case ID	Ground Truth	Predicted	Error Corridor
PV2-001	CoT/simple	hybrid/generative	CoT $\rightarrow$ hybrid
PV2-011	code/simple	hybrid/generative	code $\rightarrow$ hybrid
PV2-013	code/simple	CoT/simple	code $\rightarrow$ CoT
PV2-020	code/simple	code/complex	complexity over-est.
PV2-021	CoT/complex	hybrid/generative	CoT $\rightarrow$ hybrid
PV2-033	code/complex	hybrid/agentic	code $\rightarrow$ hybrid
PV2-034	code/complex	hybrid/agentic	code $\rightarrow$ hybrid
PV2-036	code/complex	hybrid/agentic	code $\rightarrow$ hybrid
PV2-042	hybrid/agentic	CoT/complex	hybrid $\rightarrow$ CoT
PV2-054	hybrid/generative	code/complex	hybrid $\rightarrow$ code
PV2-055	hybrid/generative	CoT/complex	hybrid $\rightarrow$ CoT
PV2-057	hybrid/generative	code/complex	hybrid $\rightarrow$ code
PV2-059	hybrid/generative	CoT/complex	hybrid $\rightarrow$ CoT

Error corridors summarize the directional confusion pattern. The three dominant corridors—code $\rightarrow$ hybrid ($n = 4$), hybrid $\rightarrow$ CoT ($n = 3$), and hybrid $\rightarrow$ code ($n = 2$)—account for 9 of 13 errors.

Table 9: Pre-registered artifacts for Study 2, committed before experiment execution.

Artifact	File Path	Purpose
Experiment design	`paper/main.tex` (§4)	Protocol specification
Analysis script	`scripts/ab-test/rct_cross_arm_analysis.py`	Endpoint analysis
Decision matrix	Table 4 (this paper)	Outcome $\rightarrow$ action map
Hypothesis specs	H5–H9 (§1.3)	Testable predictions
Corpus (60-case)	`data/progressive_test_cases_v2_60.jsonl`	Evaluation corpus
Benchmark script	`scripts/benchmark_openweights.py`	Model evaluation driver

All artifacts are version-controlled in the `micoverde/plexor-slm-frontdoor-rct` repository. Repository tag `rct-baseline-2026-03-12` marks the pre-registration snapshot.